

Construye agentes de IA con Pydantic AI

Un mapa de los conceptos, comandos y decisiones del taller. Úsalo junto al Google Colab: el notebook contiene las prácticas ejecutables.

Jueves 24 de septiembre de 2026 • 1:00–3:00 p. m.

Ricardo Tovar • Hackathon FISC

Al finalizar: podrás crear un agente sencillo, darle una herramienta, pedir una salida tipada, continuar una conversación con historial y explicar cuándo conviene añadir un reviewer.

Ruta de dos horas

Bloque	Tiempo	Qué harás
Fundamentos	10 min	Reconocer agente, tool, MCP, skill y memoria.
Colab	40 min	Construir y modificar un agente con Pydantic AI.
Patrones y demo	20 min	Comparar un agente con writer/reviewer y observar un caso real.
Coding agent	10 min	Preparar un proyecto y la skill oficial.
Construcción y cierre	40 min	Prototipo, caso ambiguo y mejora.

Necesitas Python básico, una cuenta Google para Colab y una clave Gemini propia. Guarda la clave en Colab Secrets; nunca la pegues en el notebook compartido.

Un agente toma decisiones dentro de límites

Recibe una petición, observa el contexto, puede usar herramientas y produce un resultado. Una ejecución confiable también define cómo detenerse y cómo verificar la respuesta.

Término	Qué significa aquí	Ejemplo
Tool	Acción concreta que el agente puede invocar.	Consultar un dato de un diccionario.
MCP	Protocolo para conectar herramientas y contexto externo.	Conectar un servidor de recursos.
Skill	Guía o capacidad empaquetada para un agente; el uso exacto cambia entre productos.	Instrucciones de Pydantic AI para un coding agent.
Memoria	Estado útil que se conserva o se recupera.	Historial de mensajes; después, almacenamiento seleccionado.

Premisa de Pydantic AI: construir aplicaciones de IA en Python con tipos, salida estructurada validada y herramientas definidas como funciones. El framework permite elegir el control del flujo: empieza por un agente y añade coordinación cuando la tarea lo demande.

Cuatro decisiones antes de sumar agentes

- ¿Qué salida concreta se necesita y cómo se valida?
- ¿Qué herramienta hace falta y con qué permisos?
- ¿Qué información debe durar entre turnos?
- ¿Cómo sabrás que un segundo agente mejora el resultado?

Fuentes: pydantic.dev/docs/ai/overview/ · pydantic.dev/docs/ai/guides/multi-agent-applications/

Del Colab al proyecto local

En Colab

```
%pip install -q pydantic-ai  
  
# Configura GEMINI_API_KEY en Colab Secrets.  
# Ejecuta las celdas del notebook en orden.
```

El recorrido es: **Agent** → **tool** → **output_type** → **message_history**. Cambia una sola variable a la vez. Prueba después un dato ambiguo y observa el resultado.

En un proyecto local

```
mkdir mi-agente && cd mi-agente  
uv init  
uv add pydantic-ai  
npx skills add pydantic/skills
```

El instalador de skills puede pedir destino. Elige instalación de proyecto para que las instrucciones viajen con el repositorio y revisa los archivos que añadió. Comando oficial: pydantic.dev/docs/ai/overview/coding-agent-skills/

Brief para el coding agent: "Crea un agente que extraiga acuerdos de un texto. Define un modelo Pydantic para decisiones, responsables y fechas. Usa una tool sólo para consultar datos disponibles. Incluye un caso normal, uno ambiguo y criterios de aceptación. No inventes responsables."

Revisión humana mínima

- ¿El código ejecuta con una clave propia y sin secretos guardados?
- ¿La tool devuelve hechos y la salida está validada?
- ¿El caso ambiguo produce una aclaración o una ausencia explícita?

Una idea pequeña, una prueba útil

A · Acuerdos de reunión

Extrae decisiones, responsables y pendientes de un texto corto. Añade un reviewer sólo si puedes comparar errores antes y después.

B · Guía FISC

Responde preguntas sobre un documento breve proporcionado por el facilitador. Cita el fragmento usado y admite cuando no hay información.

C · Incidencias

Clasifica solicitudes por categoría y urgencia. Pide una aclaración cuando faltan datos.

Qué entregar

Una ejecución reproducible, un caso ambiguo, un fallo observado y una mejora concreta. Una interfaz es opcional.

Recursos

- Notebook y README: github.com/rtovardev/taller-agentes-pydantic-ai-fisc-2026
- Documentación: pydantic.dev/docs/ai/overview/
- Patrones: pydantic.dev/docs/ai/guides/multi-agent-applications/
- Skill oficial: pydantic.dev/docs/ai/overview/coding-agent-skills/

Repite la prueba difícil antes de añadir otra herramienta o agente. Guarda el resultado esperado y el observado: esa comparación te dirá qué mejorar.

Uso: puedes descargar este brief para consulta y estudio personal. La redistribución, adaptación para otra charla o publicación requiere autorización de Ricardo Tovar. Consulta el aviso del repositorio.